



Rapport SAÉ S4

Base de Données — Modèle EA

Jupiter E vs Jupiter B : Comparaison des modèles de suivi de colis

GROUPE (S3) : JUPITER E

GROUPE(S4) : JUPITER 4

TABELLOUT Amir Khaled

CAV Boran

LACENE-NECER Anas

BENHADDI Amira

Année universitaire 2025-2026



1. Description globale du sujet

On travaille sur une application web de suivi de colis l'Université Sorbonne Paris Nord. Le but est simple : quand un département a besoin de commander du matériel, l'application doit permettre de suivre cette commande de A à Z, de la demande initiale jusqu'à la réception du colis et le paiement du fournisseur.

Cinq types d'utilisateurs se partagent le travail. Les agents de département créent les commandes, le service financier les valide et gère les fournisseurs, le directeur signe les bons de commande, le CRIT réceptionne et distribue les colis, et le superadmin gère les comptes et a un oeil sur tout.

Au semestre précédent, notre groupe J.E (Jupiter E) avait développé une première version en PHP vanilla avec 6 tables dans MariaDB. Ce semestre, on repart sur le projet du groupe J.B (Jupiter B), réalisé avec Laravel 12 : 8 tables, un vrai système de permissions, et Docker pour le déploiement. Ce rapport présente notre modèle de départ (J.E), puis le modèle finalisé que l'on adopte (J.B), en justifiant chaque changement par rapport à notre point de départ.

2. Description des données à stocker

Voici les différentes catégories d'informations que la base doit stocker pour que le suivi de commandes fonctionne.

Utilisateurs et droits d'accès

On stocke les infos de base de chaque utilisateur : nom, prénom, email, téléphone. Chez J.E, chacun avait un mot de passe hashé (bcrypt) et un rôle fixe sous forme d'ENUM. Chez J.B, plus besoin de mot de passe puisque l'authentification passe par le CAS de l'université. Et au lieu d'un rôle unique, on a un système de rôles et permissions séparés, avec 16 permissions possibles. C'est plus souple, un même utilisateur peut avoir plusieurs rôles.

Fournisseurs

Les fournisseurs sont enregistrés avec leurs coordonnées et leur SIRET. Chez J.E, on avait aussi leur adresse postale complète et leurs coordonnées bancaires (IBAN, BIC). Chez J.B, ces champs manquent, c'est un point à améliorer. Par contre, J.B ajoute un booléen `is_valid` qui permet de bloquer un fournisseur sans le supprimer.

Commandes

C'est la table la plus importante. Chaque commande a un numéro unique, un demandeur, un département et un fournisseur. On enregistre aussi le statut (10 chez J.E, 13 chez J.B), les documents associés (devis, bon de commande, bon de livraison) et les montants. Sur ce dernier point, il y a une vraie différence : chez J.E on séparait HT, TVA et TTC, chez J.B il n'y a qu'un montant global.

Articles / Colis

Chez J.E, une commande contenait des articles, autrement dit des lignes de commande avec désignation, quantité, prix unitaire et TVA. Chez J.B c'est différent : on parle de colis

(packages), avec un coût, un délai prévu et une date de livraison. En gros, J.E pensait en termes de facture, J.B pense en termes de livraison physique.

Suivi et traçabilité

Chez J.E, une table historique_commandes enregistrerait chaque changement de statut avec l'ancien et le nouveau statut, qui l'a fait, quand, un commentaire, et même l'adresse IP. Chez J.B, ce suivi est séparé en deux : les logs (journal automatique) et les commentaires (messages des utilisateurs). L'IP n'est plus tracée par contre.

Pièces jointes

Chez J.E, les pièces jointes avaient leur propre table avec des catégories (devis, bon de commande signé, facture, etc.), on pouvait en attacher autant qu'on voulait. Chez J.B, c'est plus simple mais plus limité : les chemins des fichiers sont stockés directement dans la table commande (path_quote, path_purchase_order, path_delivery_note). Si on veut ajouter un autre type de document, il faut modifier le schéma.

3. Modèle EA de J.E (point de départ)

Le modèle EA de J.E est notre point de départ. On y retrouve 6 entités et 7 associations qui couvrent tout le cycle de vie d'une commande. Le diagramme complet est à l'avant dernière page (MEA de départ). On détaille ici chaque entité avec ses attributs, puis les associations entre elles.

Entités

Entité : UTILISATEUR

Attribut	Type	PK	FK	NULL
<u>id</u>	INT AUTO_INCREMENT	X		Non
email	VARCHAR(100) UNIQUE			Non
password_hash	VARCHAR(255)			Non
nom	VARCHAR(50)			Non
prenom	VARCHAR(50)			Non
role	ENUM(agent,sf,directeur,crit,superadmin)			Non
departement	VARCHAR(50)			Oui
actif	BOOLEAN DEFAULT TRUE			Non
date_creation	DATETIME			Non
derniere_connexion	DATETIME			Oui

Entité : FOURNISSEUR

Attribut	Type	PK	FK	NULL
<u>id</u>	INT AUTO_INCREMENT	X		Non
nom	VARCHAR(100) UNIQUE			Non
email	VARCHAR(100)			Oui
telephone	VARCHAR(20)			Oui
adresse	VARCHAR(255)			Oui
code_postal	VARCHAR(10)			Oui
ville	VARCHAR(100)			Oui
pays	VARCHAR(50) DEFAULT 'France'			Oui
iban	VARCHAR(34)			Oui
bic	VARCHAR(11)			Oui
siret	VARCHAR(14)			Oui
actif	BOOLEAN DEFAULT TRUE			Non

Entité : COMMANDE

Attribut	Type	PK	FK	NULL
<u>id</u>	INT AUTO_INCREMENT	X		Non
numero_commande	VARCHAR(20) UNIQUE			Non
demandeur_id	INT		FK	Non
fournisseur_id	INT		FK	Non
statut	ENUM (10 valeurs)			Non
description	TEXT			Oui
urgent	BOOLEAN DEFAULT FALSE			Non
montant_ht	DECIMAL(10,2)			Oui
montant_tva	DECIMAL(10,2)			Oui
montant_ttc	DECIMAL(10,2)			Oui
commentaire_refus	TEXT			Oui
dates workflow (x7)	DATETIME			Oui

Entité : ARTICLE

Attribut	Type	PK	FK	NULL
<u>id</u>	INT AUTO_INCREMENT	X		Non
commande_id	INT		FK	Non
designation	VARCHAR(255)			Non
quantite	INT DEFAULT 1			Non
prix_unitaire	DECIMAL(10,2)			Non
tva_taux	DECIMAL(5,2) DEFAULT 20			Non
total_ttc	DECIMAL(10,2)			Non

Entité : PIECE_JOINTE

Attribut	Type	PK	FK	NULL
<u>id</u>	INT AUTO_INCREMENT	X		Non
commande_id	INT		FK	Non
nom_fichier	VARCHAR(255)			Non
type_mime	VARCHAR(100)			Non
categorie	ENUM (6 valeurs)			Non
chemin_fichier	VARCHAR(500)			Non
taille_octets	INT			Oui
ajoute_par	INT		FK	Non

Entité : HISTORIQUE_COMMANDE

Attribut	Type	PK	FK	NULL
id	INT AUTO_INCREMENT	X		Non
commande_id	INT		FK	Non
user_id	INT		FK	Non
ancien_statut	VARCHAR(30)			Non
nouveau_statut	VARCHAR(30)			Non
commentaire	TEXT			Oui
date_action	DATETIME			Non
ip_utilisateur	VARCHAR(45)			Oui

Associations

Entité 1	Entité 2	Association	Card. E1	Card. E2
UTILISATEUR	COMMANDE	<i>Demander</i>	1,N	1,1
FOURNISSEUR	COMMANDE	<i>Fournir</i>	0,N	1,1
COMMANDE	ARTICLE	<i>Contenir</i>	0,N	1,1
COMMANDE	PIECE_JOINTE	<i>Attacher</i>	0,N	1,1
UTILISATEUR	PIECE_JOINTE	<i>Ajouter</i>	0,N	1,1
COMMANDE	HISTORIQUE	<i>Tracer</i>	0,N	1,1
UTILISATEUR	HISTORIQUE	<i>Effectuer</i>	0,N	1,1

4. Modèle EA finalisé (J.B adopté)

Le modèle qu'on adopte pour la suite est celui du groupe J.B (Jupiter B), réalisé avec Laravel 12. Il vise les mêmes objectifs que notre J.E, mais la structure de la base de données est assez différente. Le diagramme est à la dernière page (MEA Finalisé). On détaille ci-dessous chaque changement par rapport à notre modèle de départ, avec nos justifications.

Changements par rapport au modèle de départ (J.E)

1. Rôles et permissions découplés. Chez J.E, le rôle était juste un ENUM dans la table UTILISATEUR — un utilisateur, un rôle, pas de nuance. Le modèle J.B découple ça complètement : les rôles et permissions deviennent des entités séparées, reliées par deux tables pivots (ROLE_USER et PERMISSION_ROLE). On peut maintenant attribuer plusieurs rôles à un même utilisateur et gérer 16 permissions fines (create_order, validate_order, sign_order, etc.). On adopte ce changement car la relation N:M entre UTILISATEUR-RÔLE puis RÔLE-PERMISSION donne une souplesse que notre ENUM ne permettait pas.

2. Authentification CAS au lieu du mot de passe. Chez J.E, chaque utilisateur avait son password_hash en bcrypt. Le modèle J.B délègue l'authentification au CAS de l'université (Central Authentication Service) : l'attribut password_hash disparaît au profit d'un login, l'identifiant CAS unique. On adopte ce choix car on n'a plus à gérer le stockage de mots de passe, et ça colle directement à l'infra existante de Paris 13.

3. COLIS au lieu d'ARTICLES. Chez J.E, la table ARTICLE représentait les lignes de commande : désignation, quantité, prix unitaire, TVA. Le modèle J.B remplace ça par COLIS (packages), qui modélise les unités physiques de livraison avec un coût, un délai prévu et une date d'expédition. C'est un changement de point de vue : J.E raisonnait en lignes de facture, J.B raisonne en colis physiques. On garde ce choix car il correspond mieux au suivi logistique qu'on veut faire. La clé primaire composite (id, order_id) identifie chaque colis dans sa commande.

4. Audit scindé en deux tables. Chez J.E, tout passait par une seule table HISTORIQUE_COMMANDE : ancien statut, nouveau statut, IP, commentaire. Le modèle J.B sépare ça en deux entités : LOG pour le journal technique des actions, et COMMENTAIRE pour les messages des utilisateurs. On valide cette séparation car c'est plus clair — on ne mélange plus les traces automatiques et les remarques humaines. Par contre, le tracking d'IP qu'on avait chez J.E n'est pas repris, c'est un point qu'on pourrait réintroduire.

5. Pièces jointes simplifiées. Chez J.E, les pièces jointes avaient leur propre table avec des catégories (devis, facture, BC signé...), la taille du fichier et le type MIME. Le modèle J.B stocke les chemins directement dans la table COMMANDE (path_quote, path_purchase_order, path_delivery_note). C'est plus simple, mais moins extensible — pour ajouter un nouveau type de document, il faudrait modifier le schéma. On accepte ce compromis pour l'instant, mais c'est un axe d'amélioration futur.

6. Workflow plus détaillé. J.E avait 10 statuts, le modèle J.B en a 13. Il ajoute une étape de devis (DEVIS, DEVIS_REFUSÉ), des refus à plusieurs niveaux (BON_DE_COMMANDE_REFUSÉ, COMMANDE_REFUSÉE), la livraison partielle (PARTIELLEMENT_LIVRÉ) et l'annulation (ANNULÉ). On adopte ce workflow enrichi car il colle mieux à la réalité des achats à l'université, où un devis peut être rejeté sans que ça annule toute la procédure.

7. Un seul montant au lieu de trois. Chez J.E, on avait montant_ht, montant_tva et montant_ttc dans COMMANDE, plus prix_unitaire, tva_taux et total_ttc dans chaque ARTICLE. Le modèle J.B ne garde qu'un seul attribut cost (DECIMAL 12,2) dans COMMANDE et dans COLIS. C'est une perte de détail comptable par rapport à notre approche, et c'est le point qu'on considère le moins positif. On le garde pour l'instant mais on prévoit de réintroduire le découpage HT/TVA/TTC.

Entités du modèle finalisé

Entité : UTILISATEUR

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
login	VARCHAR UNIQUE			Non
last_name	VARCHAR			Non
first_name	VARCHAR			Non
email	VARCHAR			Oui
phone_number	VARCHAR			Oui

Entité : ROLE

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
name	VARCHAR UNIQUE			Non
description	TINYTEXT			Non
is_department	BOOLEAN			Non

Entité : PERMISSION

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
name	VARCHAR UNIQUE			Non

Entité : FOURNISSEUR (suppliers)

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
company_name	VARCHAR UNIQUE			Non
siret	VARCHAR(14) UNIQUE			Non
email	VARCHAR			Oui
phone_number	VARCHAR			Oui
contact_name	VARCHAR			Oui
speciality	VARCHAR			Oui
note	TEXT			Oui

is_valid	BOOLEAN DEFAULT FALSE			Non
----------	-----------------------	--	--	-----

Entité : COMMANDE (orders)

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
order_num	VARCHAR UNIQUE			Non
title	VARCHAR			Non
description	TEXT			Oui
supplier_id	BIGINT		FK	Oui
cost	DECIMAL(12,2)			Oui
quote_num	VARCHAR UNIQUE			Non
status	ENUM (13 valeurs)			Non
path_quote	VARCHAR			Oui
path_purchase_order	VARCHAR			Oui
path_delivery_note	VARCHAR			Oui
author_id	BIGINT		FK	Non
department_id	BIGINT		FK	Non

Entité : COLIS (packages)

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
<u>order_id</u>	BIGINT	X	FK	Non
name	VARCHAR			Non
cost	DECIMAL(12,2)			Oui
expected_delivery_time	VARCHAR			Oui
shipping_date	DATETIME			Oui

Entité : COMMENTAIRE (comments)

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
<u>order_id</u>	BIGINT	X	FK	Non
content	TEXT			Non
author_id	BIGINT		FK	Oui

Entité : LOG (logs)

Attribut	Type	PK	FK	NULL
<u>id</u>	BIGINT AUTO_INCREMENT	X		Non
<u>order_id</u>	BIGINT	X	FK	Non
content	TEXT			Non
author_id	BIGINT		FK	Oui

Tables pivots**Entité : ROLE_USER (pivot)**

Attribut	Type	PK	FK	NULL
<u>user_id</u>	BIGINT	X	FK	Non
<u>role_id</u>	BIGINT	X	FK	Non

Entité : PERMISSION_ROLE (pivot)

Attribut	Type	PK	FK	NULL
<u>role_id</u>	BIGINT	X	FK	Non
<u>permission_id</u>	BIGINT	X	FK	Non

Associations

Entité 1	Entité 2	Association	Card. E1	Card. E2
UTILISATEUR	ROLE	<i>Posseder</i>	1,N	0,N
ROLE	PERMISSION	<i>Avoir</i>	0,N	0,N
UTILISATEUR	COMMANDE	<i>Creer</i>	1,N	1,1
ROLE (dept)	COMMANDE	<i>Departement</i>	0,N	1,1
FOURNISSEUR	COMMANDE	<i>Fournir</i>	0,N	0,1
COMMANDE	COLIS	<i>Contenir</i>	0,N	1,1
COMMANDE	COMMENTAIRE	<i>Recevoir</i>	0,N	1,1
COMMANDE	LOG	<i>Journaliser</i>	0,N	1,1
UTILISATEUR	COMMENTAIRE	<i>Ecrire</i>	0,N	0,1
UTILISATEUR	LOG	<i>Generer</i>	0,N	0,1

5. Schéma relationnel et forme normale

On traduit ici le modèle EA finalisé en schéma relationnel. Les clés primaires sont soulignées, les clés étrangères sont précédées d'un #.

Schéma relationnel du modèle finalisé

UTILISATEUR (id, login, last_name, first_name, email, phone_number)

ROLE (id, name, description, is_department)

PERMISSION (id, name)

ROLE_USER (#user_id, #role_id)

PERMISSION_ROLE (#role_id, #permission_id)

FOURNISSEUR (id, company_name, siret, email, phone_number, contact_name, speciality, note, is_valid)

COMMANDE (id, order_num, title, description, #supplier_id, cost, quote_num, status, path_quote, path_purchase_order, path_delivery_note, #author_id, #department_id)

COLIS (id, #order_id, name, cost, expected_delivery_time, shipping_date)

COMMENTAIRE (id, #order_id, content, #author_id)

LOG (id, #order_id, content, #author_id)

Forme normale

On a vérifié que le schéma est en troisième forme normale (3FN). Voici pourquoi, niveau par niveau.

1FN : Tous les attributs sont atomiques. Pas d'attribut multi-valué, pas de groupe répétitif. Les ENUM (comme le statut de commande) sont des domaines finis, pas des attributs composés.

2FN : Les tables à clé primaire simple (UTILISATEUR, RÔLE, PERMISSION, FOURNISSEUR, COMMANDE) sont en 2FN par définition. Pour les tables à clé composite — COLIS, COMMENTAIRE et LOG avec PK (id, order_id), et les pivots ROLE_USER et PERMISSION_ROLE — chaque attribut non-clé dépend de la totalité de la clé. Par exemple dans COLIS, l'attribut name dépend du couple (id, order_id), pas de l'un ou l'autre séparément.

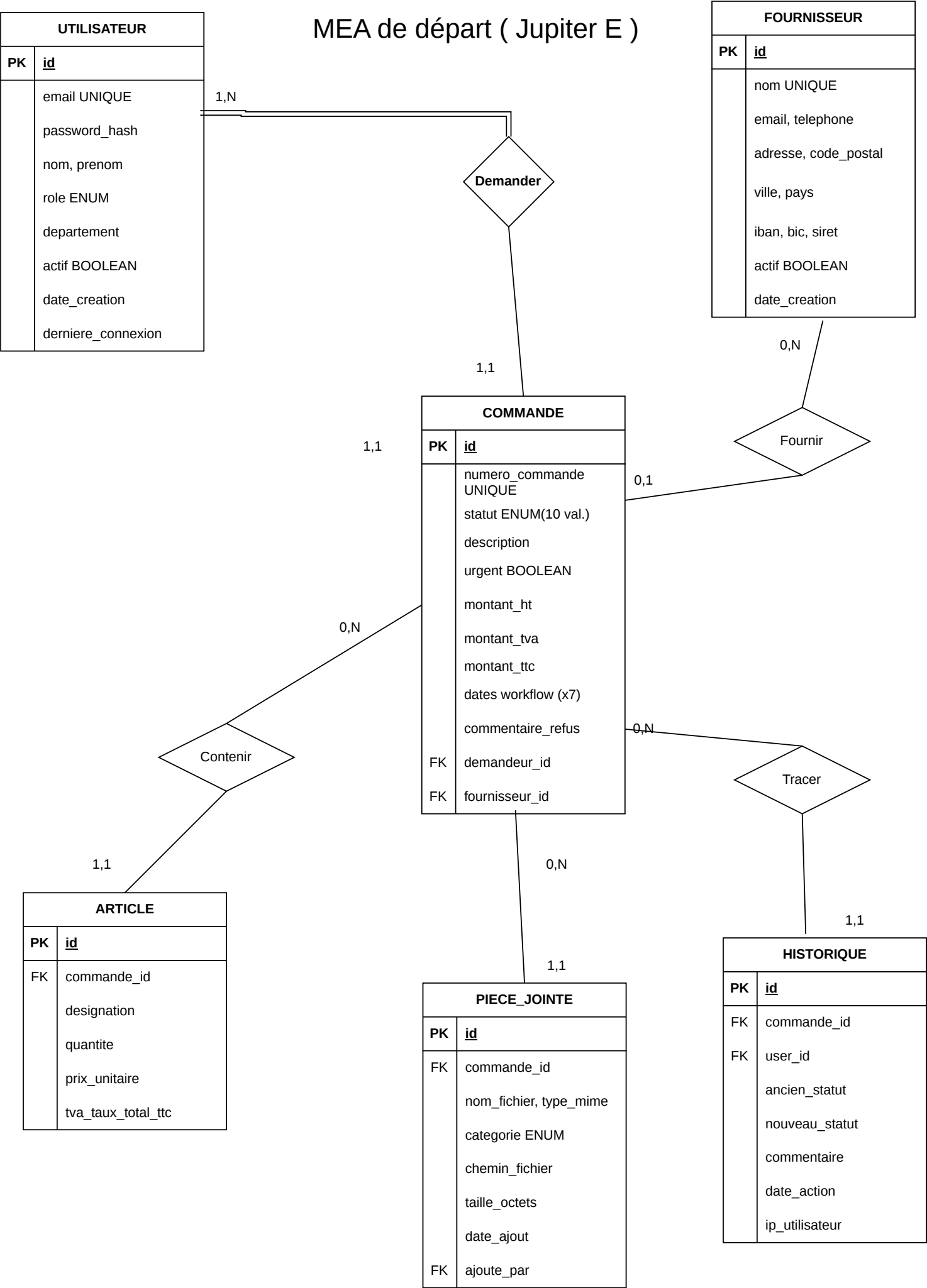
3FN : Aucune dépendance fonctionnelle transitive entre attributs non-clé. Dans COMMANDE, author_id et department_id sont des FK indépendantes qui ne déterminent pas d'autres attributs non-clé de la table. Dans FOURNISSEUR, company_name et siret sont des clés candidates (UNIQUE) mais ne créent pas de transitivité entre les autres attributs.

6. Bilan d'implication

Le tableau ci-dessous montre qui a fait quoi dans le rapport. Les pourcentages donnent la contribution de chacun par section (chaque colonne fait 100%).

Membre	Desc. sujet	Desc. BD	EA Dép. J.E	EA Final. J.B	Schéma rel.	Mise en forme
TABELLOUT Amir K.	30%	30%	30%	30%	30%	40%
CAV Boran	25%	25%	30%	30%	30%	20%
LACENE-NECER Anas	25%	25%	30%	30%	30%	20%
BENHADDI Amira	20%	20%	10%	10%	10%	20%
TOTAL	100%	100%	100%	100%	100%	100%

MEA de départ (Jupiter E)



MEA Finalisé (Jupiter B adopté)

